Dashboards and Alerting

page on symptoms, diagnose with causes

Watch the 4-min video

In one line: page a human only for urgent, actionable, user-facing symptoms, ideally an SLO burn rate, and link a runbook; everything else becomes a ticket or a dashboard panel, because a pager that cries wolf gets ignored when the real outage comes.

1 Paged at 3 a.m. for nothing

Your phone buzzes: CPU at 85% on one server. Users are fine. It happens the next night, and the next. By the time a real outage arrives, the team has learned to ignore the pager. Alert fatigue isn't a people problem; it's a design problem.

2 Symptoms, not causes

	Symptom	Cause
Examples	checkout errors, p99 above target, failed logins	high CPU, growing queue, one replica down, disk 80%
Hurts users?	always	maybe never (redundancy, headroom)
Use for	**paging**	**diagnosis** on dashboards; tickets if trending to trouble

Users don't care about your CPU; they care whether checkout works. One exception: causes that predict certain, imminent harm (disk full in 2 hours) can deserve a page.

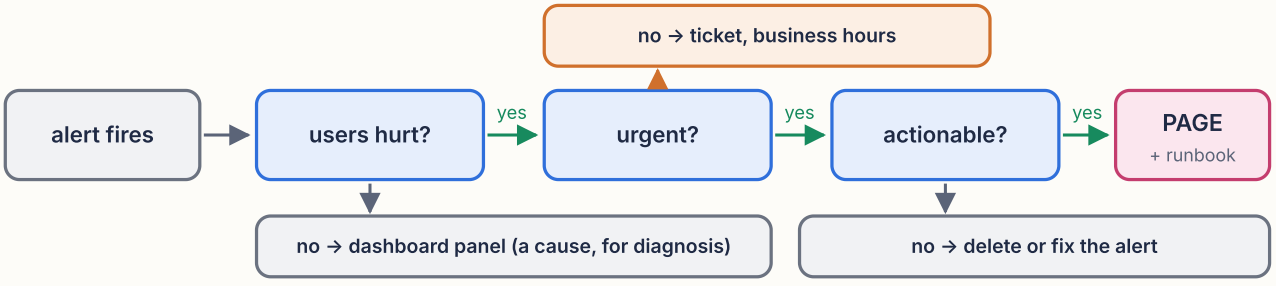


Figure 1. Three questions decide whether an alert may wake someone. Most "no" answers mean ticket, dashboard or delete.

3 SLO burn-rate alerts

A fixed threshold ("errors > 1%") is either too noisy or too slow. Instead alert on **burn rate**: how fast you're spending the error budget (lessons 2–3). Burn rate 1 spends the 30-day budget in exactly 30 days. With a 99.9% SLO the budget is 0.1% errors, so burn 14.4 means a 1.44% error rate, spending **2% of the month's budget in one hour**: page now.

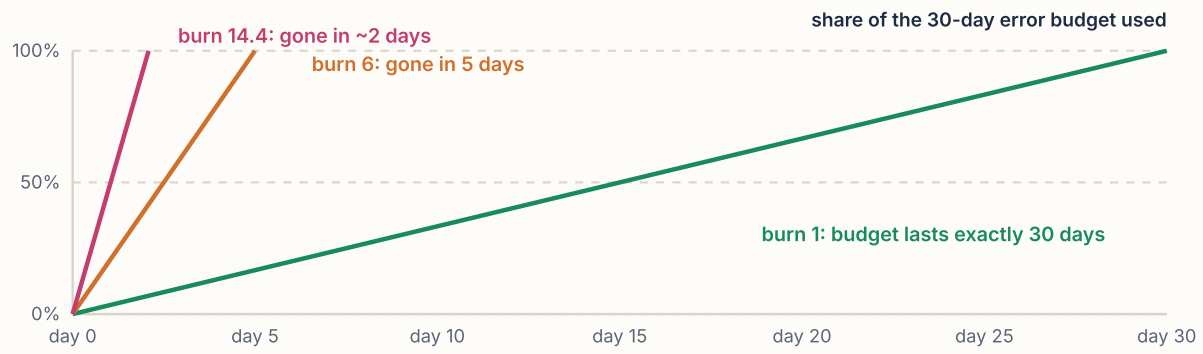


Figure 2. Budget consumption over a 30-day window for different burn rates. Time to exhaustion = 30 days ÷ burn rate.

Severity	Long window	Short window	Burn rate	Budget spent
Page	1 hour	5 minutes	14.4	2%
Page	6 hours	30 minutes	6	5%
Ticket	3 days	6 hours	1	10%

These are the Google SRE Workbook's recommended multiwindow pairs. The alert fires only if **both** windows exceed the rate: the long window proves it's significant, the short window proves it's *still happening*, so the alert also resets quickly after a fix. The result is fast and quiet.

4 Page vs ticket, and runbooks

Page only for urgent, user-facing problems that need action now; everything else is a ticket for business hours. If an alert fires and nobody needs to do anything, **delete it**. Every paging alert links a **runbook**: what the alert means, how to check impact, first mitigation steps (roll back, flag off, fail over), and who to escalate to. Review runbooks after each incident (lesson 70) and fix the steps that were wrong.

5 Dashboards are for diagnosis

- **Top row:** the four golden signals for the service: traffic, errors, latency (p50/p99), saturation.
- **Below:** dependencies (calls out, their errors and latency), then resources (CPU, memory, pools, queues).
- **One overview per service,** same layout everywhere, templated by service and region (lesson 62).
- **Link panels** to matching traces and logs; delete dashboards nobody opens.

WORKED EXAMPLE: CHECKOUT PAGES ONLY ON SLO BURN

Checkout has a **99.9% success SLO**. Rule 1: burn rate > 14.4 over **both** the last 1 h and the last 5 min (error ratio > 1.44%) → page on-call. Rule 2: burn rate > 1 over 6 h → open a ticket. Both alerts link the checkout runbook and dashboard. When one flaky node hits 95% CPU, nobody is paged: the load balancer routes around it and customers aren't failing. The CPU panel on the dashboard is where you'll look if checkout's burn rate ever does rise.

COMMON MISTAKES

- Paging on causes (CPU, memory, one pod restart) that don't hurt users.
- Alerts with no runbook link, so the 3 a.m. responder starts from zero.
- Single short window thresholds that flap on every blip; or only long windows that page hours late.
- Never deleting alerts: each one adds noise, and the team silences the channel.

6 Key terms

Symptom alert: fires on what users experience

Burn rate: rate of error-budget spend; 1 = exactly on budget

Multiwindow alert: long and short windows must both exceed the rate

Runbook: steps for responding to a specific alert

Alert fatigue: so many noisy alerts that people ignore them

Golden signals: latency, traffic, errors, saturation

7 Interview questions

Q1. What makes a good alert? BASIC

It fires on a user-facing symptom, ideally an SLO burn rate; it's urgent and actionable; and it links a runbook. Everything else becomes a ticket or a dashboard panel.

Q2. What is alert fatigue? BASIC

Too many noisy or non-actionable alerts, until people start ignoring pages, including the real one. Fix it by deleting or demoting alerts nobody acts on.

Q3. Your 99.9% service has a 0.5% error rate for the last hour. Page? SCENARIO

Burn rate = 0.5% ÷ 0.1% = 5: below both paging rates (14.4 and 6), so no page. It's far above 1, so the ticket rule fires if it persists; at this rate the month's budget is gone in 6 days, so treat the ticket as urgent.

Q4. On-call gets 40 pages a week. How do you fix it? SCENARIO

Review every alert: keep symptom alerts with runbooks, convert cause alerts to dashboards or tickets, move to burn-rate rules, and delete what nobody acted on.

Q5. Why use two windows instead of one? TRICKY

The long window shows enough budget burned to matter; the short window confirms it's still happening, so you page fast on real problems and stop soon after recovery.

Q6. Should 'disk 90% full' ever page? TRICKY

Only if it predicts certain user impact soon (e.g. full within hours at current growth); otherwise it's a ticket. Alert on time-to-full, not the percentage.

Next: SD-N-010 · Health Endpoints backendroadmap.com